AI-Powered Development Workshop: Complete Step-by-Step Guide

MSc Data Science Hands-On Workshop
From Terminal AI to IoT Data Pipelines

Table of Contents

1. Workshop Overview
2. Part 1: AI in the Terminal with Gemini CLI
3. Part 2: Docker for Reproducible Environments
4. Part 3: GitHub SSH Authentication
5. Part 4: Arduino IoT Sensor Project
6. Part 5: Python Serial Communication
7. Part 6: Weather API Integration
8. Part 7: Containerizing Your Application
9. Part 8: Git Workflow and Collaboration
10. Optional Extensions
11. Troubleshooting Guide
12. Resources for Continued Learning

Workshop Overview

What You Will Build

In this hands-on workshop, you will build a complete IoT data pipeline that integrates four powerful technologies into one cohesive professional workflow. By the end of this

workshop, you will have created a system that collects real-time sensor data from an Arduino, processes it with Python, visualizes it with matplotlib, containerizes the entire application with Docker, and manages the codebase with Git and GitHub—all while leveraging AI assistance directly in your terminal.

This is not just a technical exercise. This workshop teaches you the professional skills that appear in **87% of data science job postings** and are essential for collaborative research and industry work. You will learn to work like a professional data scientist, using industry-standard tools and workflows that eliminate common problems like "it works on my machine" and context loss when using AI assistants.

The Four Technologies You Will Master

AI in the Terminal transforms how you code by providing persistent context and direct file access. Unlike browser-based AI tools that lose context between sessions and require constant copy-pasting, terminal AI tools like Gemini CLI maintain state across sessions, can read and write files directly, and integrate seamlessly into your development workflow. This results in a workflow that is **10 times faster** than traditional browser-based AI assistance.

Docker solves the reproducibility crisis in data science. Research shows that **70% of published studies cannot be reproduced** due to environment differences, missing dependencies, and version conflicts. Docker packages your entire environment—code, libraries, dependencies, and even the operating system—into a container that runs identically everywhere. With Docker, you achieve **100% reproducibility**, ensuring that anyone can run your analysis on any machine with identical results.

GitHub with SSH Authentication provides secure, password-free collaboration using public-key cryptography. SSH keys eliminate the need to enter passwords for every Git operation, enable secure authentication without exposing credentials, and are the industry standard for professional development. You will learn how SSH keys work, how to generate and configure them, and how to use them for seamless Git operations.

Arduino IoT Sensors bring real-world data streams into your data science workflow. You will wire sensors on a breadboard, write Arduino code to read temperature, humidity, and light levels, establish serial communication between Arduino and Python, and create real-time visualizations of sensor data. This hands-on hardware experience bridges the gap between theoretical data science and real-world IoT applications.

Why This Workshop Matters

Traditional data science education often focuses on algorithms and theory while neglecting the professional tools and workflows that make you effective in industry and research settings. This workshop fills that gap by teaching you the complete pipeline from hardware to deployment, from AI-assisted coding to reproducible containerization, from individual development to collaborative workflows.

The skills you learn today will appear repeatedly throughout your career. Docker is used by **13 million+ developers worldwide** and is the standard for deploying machine learning models in production. GitHub is the platform where **100 million+ developers** collaborate on code, and SSH authentication is how professionals secure their work. AI-assisted coding is transforming software development, and knowing how to use it effectively gives you a significant productivity advantage.

By the end of this workshop, you will have a complete project in your portfolio that demonstrates your ability to integrate hardware, software, containerization, and collaboration tools. You will have a GitHub repository with clean commit history, a Docker container that anyone can run, comprehensive documentation, and real-time visualizations of sensor data. This is the kind of project that impresses employers and demonstrates your readiness for professional data science work.

Workshop Structure and Timeline

This workshop is designed as a hands-on, follow-along experience. Each part builds on the previous one, creating a cohesive workflow from start to finish. The workshop is divided into eight main parts, with optional extensions for those who want to go further.

Part 1 (30 minutes) introduces AI in the Terminal with Gemini CLI. You will install the tool, learn how to use context files, and experience the productivity boost of terminal-based AI assistance.

Part 2 (45 minutes) covers Docker fundamentals. You will understand the reproducibility problem, learn Docker concepts (images, containers, Dockerfiles), install Docker on your system, and create your first Dockerfile.

Part 3 (30 minutes) teaches GitHub SSH authentication. You will learn how public-key cryptography works, generate SSH key pairs, add keys to GitHub, and configure Git to use SSH for authentication.

Part 4 (45 minutes) is the Arduino hardware section. You will wire sensors on a breadboard using Circuito.io for guidance, write Arduino code to read sensor data, and upload code to the Arduino board.

Part 5 (30 minutes) covers Python serial communication. You will install the PySerial library, write Python code to read Arduino data, parse sensor readings, and create real-time matplotlib visualizations.

Part 6 (30 minutes) integrates weather API data. You will obtain an API key from OpenWeatherMap, fetch weather data, combine Arduino sensor data with API data, and create comparative visualizations.

Part 7 (45 minutes) containerizes your application. You will write a Dockerfile for your Python application, build and run the Docker image, handle serial port access in containers, and verify reproducibility.

Part 8 (30 minutes) covers Git workflow and collaboration. You will initialize a Git repository, create meaningful commits, push to GitHub, create pull requests, and perform code reviews.

Optional Extensions include machine learning predictions, real-time dashboards, cloud deployment, multi-sensor networks, and generative art visualizations.

Prerequisites and Materials Needed

Software Requirements: You will need a computer running Windows $^{10}/_{11}$, macOS 10.14+, or Linux (Ubuntu 20.04+ recommended). You need administrator/sudo privileges to install software. You need a stable internet connection for downloading packages and accessing APIs. You need a modern web browser (Chrome, Firefox, Edge, or Safari) for GitHub and API key management.

Hardware Requirements: You will need an Arduino Uno or compatible board (Arduino Nano, Mega, or ESP32 also work). You need a USB cable to connect Arduino to your computer. You need a breadboard for prototyping circuits. You need a DHT11 or DHT22 temperature and humidity sensor. You need a photoresistor (light-dependent resistor) for light sensing. You need jumper wires (male-to-male and male-to-female) for connections. You need a 10kΩ resistor for the photoresistor circuit.

Accounts and API Keys: You will need a GitHub account (free tier is sufficient) at https://github.com. You need a Google account for Gemini API access at

https://ai.google.dev. You need an OpenWeatherMap API key (free tier allows 1000 calls/day) at https://openweathermap.org/api.

Knowledge Prerequisites: You should have basic Python programming experience (variables, functions, loops, libraries). You should have basic command line familiarity (navigating directories, running commands). You should have basic understanding of data structures (lists, dictionaries, arrays). No prior experience with Docker, Git, or Arduino is required—we will teach everything from scratch.

Part 1: AI in the Terminal with Gemini CLI

Understanding the Problem with Browser-Based AI

Before we dive into the solution, it is important to understand why terminal-based AI is a game-changer for developers and data scientists. Traditional browser-based AI tools like ChatGPT, Claude, and Gemini web interfaces suffer from several critical limitations that slow down your workflow and reduce productivity.

Context Loss Between Sessions is the most frustrating problem. Every time you close your browser or start a new chat, the AI forgets everything about your project. You have to re-explain your project structure, re-paste your code, and re-establish context. This wastes time and breaks your flow.

Scattered Conversations Across Multiple Tabs make it hard to maintain coherent project context. You might have one chat for Python questions, another for Docker issues, and a third for debugging. The AI cannot see the connections between these conversations, leading to inconsistent advice and repeated explanations.

No Direct File Access means you constantly copy and paste code between your editor and the browser. This is error-prone (you might paste outdated code), time-consuming (switching between windows breaks focus), and frustrating (the AI cannot see your full project structure).

Limited Control Over Workflow occurs because browser interfaces are designed for general chat, not development workflows. You cannot easily integrate AI assistance into your build process, testing pipeline, or deployment scripts. The AI is a separate tool rather than an integrated part of your development environment.

Professional Workflows Require Terminal Integration because modern development happens in the terminal. You run tests, build containers, commit code, and deploy applications all from the command line. Having your AI assistant in the same environment where you work creates a seamless, efficient workflow.

How Terminal AI Solves These Problems

Terminal-based AI tools like Gemini CLI eliminate all of these problems by integrating AI assistance directly into your development environment. Here is how they work and why they are so much better.

Persistent Context Files maintain state across sessions. When you work in a project directory, Gemini creates a context file that stores the conversation history, project structure, and relevant files. When you return to the project later, Gemini remembers everything. You never lose context, and you can pick up exactly where you left off.

Direct File System Access means Gemini can read your code files, configuration files, and data files without you copying and pasting. You can say "analyze my Python script" and Gemini reads it directly. You can say "fix the bug in my Dockerfile" and Gemini sees the entire file. This eliminates copy-paste errors and saves enormous amounts of time.

Unified Project Context keeps everything in one place. All conversations about your project happen in the same context, with access to the same files. Gemini can see how your Python code, Dockerfile, and Arduino sketch relate to each other, providing more coherent and consistent advice.

Seamless Workflow Integration allows you to use AI assistance as part of your normal development process. You can ask Gemini to generate code, then immediately test it. You can ask for debugging help while looking at error messages. You can request documentation while writing README files. The AI becomes a natural part of your workflow rather than a separate tool.

Local Control and Privacy give you more control over your data. Context files are stored locally on your machine. You can choose which files to share with the AI. You can work offline with cached context. This is especially important for sensitive research data or proprietary code.

Installing Gemini CLI

Now let us install Gemini CLI and experience the difference firsthand. The installation process is straightforward, but we will walk through each step carefully to ensure everything works correctly.

Prerequisites Check

Before installing Gemini CLI, verify that you have the necessary prerequisites installed on your system.

Node.js Version 16 or Higher is required because Gemini CLI is built with Node.js. Check your Node.js version by opening a terminal and running:

```
node --version
```

You should see output like `v16.20.0` or higher. If you do not have Node.js installed, or if your version is older than 16, download and install the latest LTS version from https://nodejs.org. The LTS (Long Term Support) version is recommended for stability.

npm (Node Package Manager) comes bundled with Node.js, but verify it is available:

```
npm --version
```

You should see a version number like `8.19.4` or higher. If npm is not found, reinstalling Node.js will include it.

Google Account is required to obtain a Gemini API key. You can use any Google account (personal or organizational). Go to https://ai.google.dev to sign in and access the API key management interface.

Terminal Access means you need a command line interface. On Windows, use PowerShell or Windows Terminal (not Command Prompt, which has limited functionality). On macOS, use Terminal.app or iTerm2. On Linux, use your default terminal emulator (GNOME Terminal, Konsole, xterm, etc.).

Obtaining a Gemini API Key

Before installing the CLI tool, you need an API key to authenticate with Google's Gemini service.

Step 1: Navigate to https://ai.google.dev in your web browser. Sign in with your Google account if you are not already signed in.

Step 2: Click on "Get API Key" or "API Keys" in the navigation menu. You will be taken to the Google AI Studio interface.

Step 3: Click "Create API Key" button. You may need to accept terms of service if this is your first time using Google AI services.

Step 4: Your API key will be generated and displayed. It looks like a long string of random characters, for example: `AIzaSyB1234567890abcdefghijklmnopqrstuvwxyz`. Copy this key immediately—you will not be able to see it again after closing the dialog.

Step 5: Store your API key securely. Do not share it publicly, do not commit it to Git repositories, and do not paste it in public forums. Treat it like a password. We will configure it in your shell environment in the next step.

Important Security Note: API keys provide access to your Google AI quota and usage. If someone obtains your API key, they can use your quota and potentially incur charges (though the free tier is generous). Keep it private.

Installing the Gemini CLI Package

Now that you have your API key, install the Gemini CLI tool globally using npm.

Step 1: Open your terminal and run the following command:

```
npm install -g @google/generative-ai-cli
```

The `-g` flag installs the package globally, making the `gemini` command available from any directory. The installation may take a minute or two as npm downloads the package and its dependencies.

Step 2: Verify the installation by checking the version:

```
gemini --version
```

You should see a version number like `1.0.0` or similar. If you see "command not found," the installation may not have completed successfully. Try closing and reopening your terminal, or check that your npm global bin directory is in your PATH.

Step 3: Configure your API key as an environment variable. This step varies by operating system and shell.

For macOS and Linux (bash shell):

Open your shell configuration file in a text editor:

```
nano ~/.bashrc
```

Add the following line at the end of the file (replace `YOUR_API_KEY_HERE` with your actual API key):

```
export GEMINI_API_KEY="YOUR_API_KEY_HERE"
```

Save the file (Ctrl+O, Enter, Ctrl+X in nano) and reload your shell configuration:

```
source ~/.bashrc
```

For macOS and Linux (zsh shell, default on modern macOS):

Open your shell configuration file:

```
nano ~/.zshrc
```

Add the same export line, save, and reload:

```
source ~/.zshrc
```

For Windows (PowerShell):

Open PowerShell as Administrator and run:

```
[System.Environment]::SetEnvironmentVariable('GEMINI_API_KEY',
'YOUR_API_KEY_HERE', 'User')
```

Close and reopen PowerShell for the change to take effect.

Step 4: Verify that your API key is configured correctly:

```
echo $GEMINI_API_KEY
```

You should see your API key printed in the terminal. If you see nothing, the environment variable was not set correctly. Review the previous step and ensure you reloaded your shell configuration.

Using Gemini CLI for the First Time

Now that Gemini CLI is installed and configured, let us use it for the first time and experience the power of terminal-based AI assistance.

Creating a Project Directory

Gemini CLI works best when you use it within a project directory. Create a directory for this workshop:

```
mkdir ~/arduino-iot-workshop
cd ~/arduino-iot-workshop
```

This creates a directory in your home folder and navigates into it. All of your workshop files will be stored here, and Gemini will create its context file in this directory.

Starting Your First Gemini Session

Launch Gemini CLI by simply typing:

```
gemini
```

You will see a welcome message and a prompt like:

```
Gemini CLI v1.0.0
Context file: /Users/yourname/arduino-iot-workshop/.gemini-context.json

You:
```

The context file path shows where Gemini is storing the conversation history. This file is created automatically in your current directory.

Your First Command: Project Planning

Let us ask Gemini to help us plan the Arduino sensor project. Type the following at the prompt:

```
Create a detailed plan for building an Arduino temperature and humidity
sensor data pipeline that sends data to Python for real-time visualization
with matplotlib.
```

Press Enter and watch Gemini generate a comprehensive plan. It will outline the hardware requirements, Arduino code structure, Python serial communication, visualization approach, and potential challenges. This demonstrates Gemini’ s ability to understand complex, multi-technology projects and provide structured guidance.

Understanding Context Files

After your first interaction, Gemini created a context file in your project directory. Let us examine it:

```
ls -la
```

You will see a file named `.gemini-context.json`. This file stores:

- **Conversation History:** All of your questions and Gemini' s responses
- **Project Context:** Information about your project structure and files
- **Session State:** Where you left off in the conversation

The beauty of context files is persistence. Close your terminal, open it again tomorrow, navigate to the same directory, and run `gemini` again. Gemini will remember your entire conversation and project context. You can continue exactly where you left off without re-explaining anything.

Asking Gemini to Read and Write Files

One of the most powerful features of terminal AI is direct file access. Let us demonstrate this by asking Gemini to create a file.

In your Gemini session, type:

```
Create a README.md file for this project that explains what we're building
and lists the hardware requirements.
```

Gemini will generate the content and write it directly to a file in your project directory. Exit Gemini (type `exit` or press Ctrl+D) and verify the file was created:

```
ls
cat README.md
```

You will see the README file that Gemini created. This eliminates the need to copy and paste code from a browser—Gemini writes files directly.

You can also ask Gemini to read files. Create a simple Python script:

```
echo "print('Hello, workshop!')" > test.py
```

Start Gemini again and ask:

```
Read test.py and suggest improvements.
```

Gemini will read the file directly from your filesystem and provide feedback. This seamless file integration is what makes terminal AI so powerful for development work.

Best Practices for Using Terminal AI

To get the most out of Gemini CLI and terminal-based AI tools, follow these best practices.

Work in Project Directories to maintain context. Always navigate to your project directory before starting Gemini. This ensures the context file is stored with your project and Gemini has access to your project files.

Use Descriptive Prompts that provide context. Instead of "fix this," say "fix the serial communication error in my Python script that reads Arduino data." The more context you provide, the better Gemini's responses will be.

Ask Gemini to Read Files instead of pasting code. Say "analyze my Arduino sketch" rather than copying and pasting the entire file into the chat. This is faster and less error-prone.

Request File Creation for boilerplate code. Ask Gemini to "create a Dockerfile for a Python application with PySerial and matplotlib dependencies" and it will generate the file directly.

Maintain Conversation Flow by building on previous questions. Since Gemini remembers context, you can say "now add error handling to that function" and it will know which function you mean.

Review Generated Code before using it. AI-generated code is usually good but not always perfect. Read through it, understand what it does, and test it before relying on it.

Use Gemini for Learning by asking "why" and "how" questions. If Gemini suggests a solution, ask "why does this approach work?" or "how does this library function?" to deepen your understanding.

Keep Context Files in Version Control (but not API keys). The `.gemini-context.json` file can be committed to Git so your team can see the AI-assisted development process. Just make sure your API key is in environment variables, not in the context file.

Part 2: Docker for Reproducible Environments

The Reproducibility Crisis in Data Science

Before we learn Docker, we need to understand the problem it solves. Data science and research face a serious reproducibility crisis. Studies show that **70% of published research cannot be reproduced** by other researchers, and one of the main reasons is environment differences.

The "It Works on My Machine" Problem

You have probably experienced this frustrating scenario. You write a Python script that works perfectly on your laptop. You share it with a colleague, and they get errors. You publish your research with code, and reviewers cannot run it. You try to run your own analysis six months later, and it breaks because you upgraded Python or a library.

This happens because of several environment-related issues:

Different Python Versions cause compatibility problems. Your script uses Python 3.9 features, but your colleague has Python 3.7. Or you used Python 3.8 and now you have Python 3.10, which deprecated a function you relied on.

Missing Libraries or Wrong Versions are extremely common. Your script requires numpy 1.21, but your colleague has numpy 1.19. Or you have scikit-learn 0.24, but the latest version is 1.0 with breaking API changes. Or you forgot to document that you installed a library, and your colleague does not know they need it.

Operating System Differences cause subtle bugs. Your script uses file paths with forward slashes (Linux/macOS), but your colleague uses Windows with backslashes. Or

your script relies on a Linux-specific library that does not exist on Windows. Or you use a library that behaves differently on different operating systems.

System Dependencies are often undocumented. Your script requires a C library that you installed months ago and forgot about. Or it needs a specific version of a system tool. Or it depends on environment variables that you set but never documented.

Inability to Reproduce Research Results damages scientific credibility. If other researchers cannot run your code and verify your results, your research is less trustworthy. Funding agencies and journals increasingly require reproducible research, and "it works on my machine" is not an acceptable answer.

Collaboration Becomes Impossible when every team member has a different environment. You spend more time debugging environment issues than actually doing data science. Onboarding new team members takes days as they try to set up the correct environment. Code reviews are difficult because reviewers cannot run the code.

This is the reproducibility crisis, and it is a serious problem in data science, machine learning, and research. Docker solves this problem completely.

How Docker Solves the Reproducibility Problem

Docker packages your entire environment—code, libraries, dependencies, and even the operating system—into a **container** that runs identically everywhere. With Docker, you achieve **100% reproducibility**. If it works in a Docker container on your machine, it will work in the same Docker container on any other machine.

Understanding Containers

A **container** is a lightweight, standalone, executable package that includes everything needed to run your application. Think of it as a complete, self-contained environment that includes:

- **Your Code:** The Python scripts, Arduino sketches, configuration files, and data files that make up your project.
- **Runtime:** The Python interpreter, Node.js runtime, or any other language runtime your code needs.
- **Libraries:** All Python packages (numpy, pandas, matplotlib, etc.) at the exact versions you specify.

- **System Tools:** Any command-line tools or utilities your code depends on.
- **Operating System:** A minimal Linux distribution that provides the foundation for everything else.

The key insight is that containers are **isolated** from the host system. Your container has its own filesystem, its own network stack, and its own process space. This means:

- **No Conflicts:** Your container can use Python 3.9 even if your host system has Python 3.10.
- **No Missing Dependencies:** Everything your code needs is inside the container.
- **No "It Works on My Machine":** The container runs identically on your laptop, your colleague's laptop, a university server, or a cloud platform.

Docker Images: The Blueprint

Before you can run a container, you need a **Docker image**. An image is a read-only template that contains the instructions for creating a container. Think of it as a blueprint or recipe.

A Docker image is built from a **Dockerfile**, which is a text file that specifies:

- **Base Image:** What operating system and base software to start with (e.g., `python:3.9-slim` for a minimal Linux with Python 3.9).
- **Dependencies:** What libraries and tools to install (e.g., `pip install numpy pandas matplotlib`).
- **Code:** What files to copy into the image (e.g., your Python scripts and data files).
- **Configuration:** What environment variables to set, what ports to expose, and what command to run when the container starts.

You build an image once, and then you can create as many containers from that image as you want. This is like baking: the recipe (Dockerfile) produces a cake mix (image), and you can bake many cakes (containers) from that mix.

Docker vs. Virtual Machines

You might be familiar with virtual machines (VMs) like VirtualBox or VMware. Docker containers are similar to VMs in that they provide isolated environments, but they are much more efficient.

Virtual Machines include a full operating system with its own kernel. Each VM runs a complete OS, which requires significant memory and disk space. VMs are slow to start (minutes) because they boot an entire operating system. VMs are heavyweight—running multiple VMs on one machine is resource-intensive.

Docker Containers share the host operating system’s kernel. They include only the application and its dependencies, not a full OS. Containers start in seconds because they do not boot an OS. Containers are lightweight—you can run dozens of containers on one machine without significant overhead.

For data science and development work, containers are almost always the better choice. They provide the isolation and reproducibility of VMs with a fraction of the resource usage and startup time.

Installing Docker

Now that you understand what Docker is and why it matters, let us install it on your system. The installation process varies by operating system, so follow the instructions for your platform.

Installing Docker on Windows

Step 1: Check Windows Version and Enable WSL 2

Docker Desktop for Windows requires Windows 10 version 1903 or higher, or Windows 11. It also requires WSL 2 (Windows Subsystem for Linux version 2), which provides a Linux kernel that Docker uses.

Check your Windows version by pressing Win+R, typing `winver`, and pressing Enter. If you have an older version, update Windows before proceeding.

Enable WSL 2 by opening PowerShell as Administrator and running:

```
wsl --install
```

This installs WSL 2 and the default Ubuntu distribution. Restart your computer when prompted.

Step 2: Download and Install Docker Desktop

Go to https://www.docker.com/products/docker-desktop and click "Download for Windows." Run the installer and follow the prompts. When asked, ensure "Use WSL 2 instead of Hyper-V" is checked.

Step 3: Start Docker Desktop

After installation, launch Docker Desktop from the Start menu. You will see a Docker icon in your system tray. Wait for Docker to start—this may take a minute or two the first time.

Step 4: Verify Installation

Open PowerShell and run:

```
docker --version
```

You should see output like `Docker version 24.0.6, build ed223bc`. Then test that Docker can run containers:

```
docker run hello-world
```

Docker will download a small test image and run it. You should see a message that says "Hello from Docker!" confirming that Docker is working correctly.

Installing Docker on macOS

Step 1: Check macOS Version

Docker Desktop requires macOS 10.14 (Mojave) or newer. Check your version by clicking the Apple menu → About This Mac. If you have an older version, consider updating macOS.

Step 2: Download Docker Desktop

Go to https://www.docker.com/products/docker-desktop and click "Download for Mac." Choose the version for your processor:

- **Apple Silicon (M1/M2/M3):** Download "Docker Desktop for Mac with Apple silicon"
- **Intel Processor:** Download "Docker Desktop for Mac with Intel chip"

Step 3: Install Docker Desktop

Open the downloaded `.dmg` file and drag the Docker icon to your Applications folder. Launch Docker from Applications. You may be prompted to enter your password to grant Docker the necessary permissions.

Step 4: Verify Installation

Open Terminal and run:

```
docker --version
```

You should see output like `Docker version 24.0.6, build ed223bc`. Then test Docker:

```
docker run hello-world
```

You should see the "Hello from Docker!" message.

Alternative: Install with Homebrew

If you use Homebrew, you can install Docker with:

```
brew install --cask docker
```

Then launch Docker from Applications.

Installing Docker on Linux (Ubuntu/Debian)

Step 1: Update Package Index

Open a terminal and update your package index:

```
sudo apt update
```

Step 2: Install Docker

Install Docker using apt:

```
sudo apt install docker.io -y
```

Step 3: Start and Enable Docker Service

Start the Docker service and enable it to start on boot:

```
sudo systemctl start docker
sudo systemctl enable docker
```

Step 4: Add Your User to the Docker Group

By default, Docker requires root privileges. To run Docker commands without `sudo`, add your user to the `docker` group:

```
sudo usermod -aG docker $USER
```

Log out and log back in for this change to take effect. Alternatively, run:

```
newgrp docker
```

Step 5: Verify Installation

Check the Docker version:

```
docker --version
```

Test Docker:

```
docker run hello-world
```

You should see the “Hello from Docker!” message.

Understanding Docker Concepts

Before we create our first Dockerfile, let us clarify the three core Docker concepts: images, containers, and Dockerfiles.

Docker Images

A **Docker image** is a read-only template that contains everything needed to run your application. It includes:

- A base operating system (usually a minimal Linux distribution)
- Your application code
- All dependencies and libraries
- Configuration files and environment variables
- Metadata about the image (author, version, exposed ports, etc.)

Images are built in layers. Each instruction in a Dockerfile creates a new layer. Layers are cached, which makes rebuilding images fast—only changed layers need to be rebuilt.

You can think of an image as a snapshot of a filesystem at a specific point in time. When you run an image, Docker creates a container from that snapshot.

Images are stored in **registries** like Docker Hub (https://hub.docker.com), where you can find pre-built images for common software (Python, Node.js, PostgreSQL, etc.) and share your own images.

Docker Containers

A **Docker container** is a running instance of an image. When you run an image, Docker creates a container with its own:

- Filesystem (based on the image, with a writable layer on top)
- Network interface (with its own IP address)
- Process space (isolated from other containers and the host)
- Resource limits (you can limit CPU, memory, and disk usage)

Containers are **ephemeral** by default. When you stop a container, any changes you made to its filesystem are lost (unless you use volumes, which we will cover later). This

is a feature, not a bug—it ensures that containers are reproducible and stateless.

You can run multiple containers from the same image. Each container is isolated from the others. For example, you could run three containers from the same Python image, each running a different script, and they would not interfere with each other.

Dockerfiles

A **Dockerfile** is a text file that contains instructions for building a Docker image. It is like a recipe or build script. Each instruction in a Dockerfile creates a new layer in the image.

Common Dockerfile instructions include:

- **FROM:** Specifies the base image to start from (e.g., `FROM python:3.9-slim`)
- **RUN:** Executes a command during the build process (e.g., `RUN pip install numpy`)
- **COPY:** Copies files from your host machine into the image (e.g., `COPY app.py /app/`)
- **WORKDIR:** Sets the working directory for subsequent instructions (e.g., `WORKDIR /app`)
- **ENV:** Sets environment variables (e.g., `ENV PYTHONUNBUFFERED=1`)
- **EXPOSE:** Documents which ports the container listens on (e.g., `EXPOSE 5000`)
- **CMD:** Specifies the command to run when a container starts (e.g., `CMD ["python", "app.py"]`)

You write a Dockerfile, build an image from it with `docker build`, and then run containers from that image with `docker run`.

The Recipe Analogy

Here is a helpful analogy to tie these concepts together:

- **Dockerfile** = Recipe with step-by-step instructions
- **Image** = Pre-made cake mix created from the recipe
- **Container** = Actual baked cake you can eat

One recipe creates one mix, but you can bake many cakes from that mix. Similarly, one Dockerfile creates one image, but you can run many containers from that image.

Creating Your First Dockerfile

Now that you understand Docker concepts, let us create a simple Dockerfile for a Python application. This will help you understand the syntax and structure before we create the more complex Dockerfile for the Arduino project.

Example: Python Hello World Dockerfile

Create a new file named `Dockerfile` (no file extension) in your project directory:

```
cd ~/arduino-iot-workshop
nano Dockerfile
```

Add the following content:

```
# Use Python 3.9 slim image as the base
FROM python:3.9-slim

# Set the working directory inside the container
WORKDIR /app

# Copy the Python script into the container
COPY hello.py /app/

# Install any dependencies (none for this simple example)
# RUN pip install --no-cache-dir some-package

# Set environment variable to prevent Python from buffering output
ENV PYTHONUNBUFFERED=1

# Specify the command to run when the container starts
CMD ["python", "hello.py"]
```

Save the file (Ctrl+O, Enter, Ctrl+X in nano).

Now create the simple Python script that this Dockerfile references:

```
nano hello.py
```

Add this content:

```
print("Hello from Docker!")
print("This Python script is running inside a container.")
```

Save the file.

Building the Docker Image

Build the image with the `docker build` command:

```
docker build -t hello-docker .
```

Let us break down this command:

- `docker build` tells Docker to build an image
- `-t hello-docker` tags the image with the name "hello-docker" (you can use any name)
- `.` specifies the build context (the current directory, where Docker looks for the Dockerfile and files to copy)

Docker will execute each instruction in the Dockerfile and show you the progress. The first time you build, it will download the base image (`python:3.9-slim`), which may take a minute. Subsequent builds will be much faster because Docker caches layers.

Running the Container

Once the image is built, run a container from it:

```
docker run hello-docker
```

You should see:

```
Hello from Docker!
This Python script is running inside a container.
```

Congratulations! You just built and ran your first Docker container. The Python script ran inside an isolated environment with its own Python installation, completely independent of your host system.

Understanding What Happened

Let us review what happened step by step:

1. **You wrote a Dockerfile** that specifies how to build an image (base image, files to copy, command to run).
2. **You built an image** with `docker build`, which executed each Dockerfile instruction and created layers.
3. **You ran a container** with `docker run`, which created an isolated environment from the image and executed the CMD instruction.
4. **The Python script ran** inside the container, printed output, and then the container stopped (because the script finished).

This is the basic Docker workflow. In the next sections, we will create a more complex Dockerfile for the Arduino project that includes multiple dependencies, serial port access, and real-time visualization.

Part 3: GitHub SSH Authentication

Understanding SSH and Public-Key Cryptography

Before we set up SSH keys for GitHub, it is important to understand what SSH is, how it works, and why it is the industry standard for secure authentication.

What is SSH?

SSH (Secure Shell) is a cryptographic network protocol that provides secure communication over an unsecured network. It was originally designed for remote login

to servers, but it is widely used for secure file transfer, command execution, and—relevant to us—Git authentication.

When you use SSH to authenticate with GitHub, you are using public-key cryptography instead of passwords. This is more secure, more convenient, and the standard practice in professional development.

How Public-Key Cryptography Works

Public-key cryptography uses a pair of keys: a **public key** and a **private key**. These keys are mathematically related but cannot be derived from each other.

The Private Key is kept secret on your computer. You never share it with anyone. It is like the key to your house—you keep it safe and do not give it to strangers.

The Public Key can be freely shared. You add it to GitHub, and GitHub uses it to verify your identity. It is like a lock—you can give copies of the lock to anyone, but only someone with the private key can unlock it.

Here is how authentication works:

1. **You generate a key pair** (one private key, one public key) on your computer.
2. **You add the public key to GitHub.** GitHub stores it in your account settings.
3. **When you push code to GitHub,** your Git client uses your private key to create a digital signature.
4. **GitHub verifies the signature** using your public key. If the signature is valid, GitHub knows the request came from someone who has the private key (you).
5. **Authentication succeeds** without ever sending your private key over the network.

This is more secure than passwords because:

- **Your private key never leaves your computer.** Even if someone intercepts the network traffic, they cannot steal your private key.
- **Brute-force attacks are infeasible.** SSH keys are 2048 or 4096 bits long, making them essentially impossible to guess.
- **You can revoke keys without changing passwords.** If your computer is compromised, you can remove the public key from GitHub without affecting your password or other keys.

Why SSH is Better Than HTTPS for Git

When you clone a Git repository, you can use either HTTPS or SSH. HTTPS requires you to enter your username and password (or a personal access token) for every push. SSH uses your key pair, so you never enter credentials.

SSH Advantages:

- **No Password Prompts:** Once your SSH key is configured, you never enter credentials again.
- **More Secure:** Private keys are much stronger than passwords.
- **Industry Standard:** Professional developers use SSH for Git authentication.
- **Easier for Automation:** Scripts and CI/CD pipelines can use SSH keys without storing passwords.

HTTPS Disadvantages:

- **Credential Prompts:** You must enter credentials for every push (or configure credential caching, which is less secure).
- **Personal Access Tokens:** GitHub deprecated password authentication for HTTPS, requiring personal access tokens instead, which are cumbersome to manage.
- **Less Secure:** Tokens can be accidentally committed to repositories or exposed in logs.

For these reasons, we will set up SSH authentication for GitHub.

Generating SSH Keys

Now let us generate an SSH key pair on your computer. The process is the same on all operating systems.

Step 1: Check for Existing SSH Keys

Before generating new keys, check if you already have SSH keys:

```
ls -la ~/.ssh
```

Look for files named `id_rsa` and `id_rsa.pub`, or `id_ed25519` and `id_ed25519.pub`. If these files exist, you already have SSH keys and can skip to the next section. If not, continue with key generation.

Step 2: Generate a New SSH Key Pair

Generate a new SSH key pair using the `ssh-keygen` command. We will use the Ed25519 algorithm, which is modern, secure, and recommended by GitHub.

Run this command (replace `your_email@example.com` with your actual email address):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

You will see output like:

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/yourname/.ssh/id_ed25519):
```

Press Enter to accept the default file location.

Next, you will be prompted for a passphrase:

```
Enter passphrase (empty for no passphrase):
```

Passphrase Decision: A passphrase adds an extra layer of security. If someone steals your private key file, they cannot use it without the passphrase. However, you will need to enter the passphrase every time you use the key (unless you use an SSH agent, which we will configure).

For this workshop, you can press Enter to skip the passphrase (no passphrase). In a professional setting, you should use a strong passphrase.

Press Enter again to confirm:

```
Enter same passphrase again:
```

The key pair will be generated:

```
Your identification has been saved in /Users/yourname/.ssh/id_ed25519
Your public key has been saved in /Users/yourname/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:abcdefghijklmnopqrstuvwxyz1234567890 your_email@example.com
```

You now have two files:

- `~/.ssh/id_ed25519` — Your private key (keep this secret!)
- `~/.ssh/id_ed25519.pub` — Your public key (you will add this to GitHub)

Step 3: Start the SSH Agent

The SSH agent is a background program that holds your private keys in memory so you do not have to enter your passphrase repeatedly. Start the agent:

```
eval "$(ssh-agent -s)"
```

You will see output like:

```
Agent pid 12345
```

This means the agent is running.

Step 4: Add Your Private Key to the SSH Agent

Add your private key to the agent:

```
ssh-add ~/.ssh/id_ed25519
```

If you set a passphrase, you will be prompted to enter it now. The agent will remember the key for the duration of your session.

On macOS, you can configure the agent to automatically load keys and store passphrases in the keychain. Create or edit `~/.ssh/config`:

```
nano ~/.ssh/config
```

Add these lines:

```
Host *
  AddKeysToAgent yes
  UseKeychain yes
  IdentityFile ~/.ssh/id_ed25519
```

Save the file. Now your key will be automatically loaded when you start your computer.

Adding Your SSH Key to GitHub

Now that you have generated an SSH key pair, you need to add the public key to your GitHub account.

Step 1: Copy Your Public Key

Display your public key:

```
cat ~/.ssh/id_ed25519.pub
```

You will see output like:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAbcdefghijklmnopqrstuvwxyz1234567890
your_email@example.com
```

Select and copy the entire output (from `ssh-ed25519` to your email address). Be careful not to include any extra spaces or line breaks.

Alternative: On macOS, you can copy the key to your clipboard with:

```
pbcopy < ~/.ssh/id_ed25519.pub
```

On Linux with xclip installed:

```
xclip -sel clip < ~/.ssh/id_ed25519.pub
```

On Windows (PowerShell):

```
Get-Content ~/.ssh/id_ed25519.pub | Set-Clipboard
```

Step 2: Add the Key to GitHub

1. Go to https://github.com and sign in to your account.
2. Click your profile picture in the top-right corner and select **Settings**.
3. In the left sidebar, click **SSH and GPG keys**.
4. Click the green **New SSH key** button.
5. In the "Title" field, enter a descriptive name for this key (e.g., "Workshop Laptop" or "Personal MacBook").
6. In the "Key" field, paste your public key (the entire output from the previous step).
7. Click **Add SSH key**.
8. GitHub may prompt you to enter your password to confirm.

Your SSH key is now added to GitHub!

Step 3: Test Your SSH Connection

Verify that your SSH key is working by testing the connection to GitHub:

```
ssh -T git@github.com
```

You may see a warning like:

```
The authenticity of host 'github.com (140.82.113.4)' can't be established.
ED25519 key fingerprint is
SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
Are you sure you want to continue connecting (yes/no)?
```

Type `yes` and press Enter. This adds GitHub to your list of known hosts.

You should then see:

```
Hi yourusername! You've successfully authenticated, but GitHub does not
provide shell access.
```

This confirms that your SSH key is working correctly. The message about "no shell access" is normal—GitHub does not allow SSH login, only Git operations.

Configuring Git to Use SSH

Now that your SSH key is set up, configure Git to use SSH URLs instead of HTTPS URLs.

Step 1: Configure Git User Information

If you have not already configured Git with your name and email, do so now:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Use the same email address that you used for your SSH key and your GitHub account.

Step 2: Clone Repositories with SSH

When cloning a repository, use the SSH URL instead of the HTTPS URL. SSH URLs look like:

```
git@github.com:username/repository.git
```

For example, to clone a repository:

```
git clone git@github.com:username/repository.git
```

Step 3: Convert Existing Repositories to SSH

If you already cloned a repository with HTTPS and want to switch to SSH, change the remote URL:

```
cd /path/to/your/repository
git remote set-url origin git@github.com:username/repository.git
```

Verify the change:

```
git remote -v
```

You should see SSH URLs (starting with `git@github.com`) instead of HTTPS URLs (starting with `https://github.com`).

Understanding SSH Key Security

SSH keys are powerful and secure, but they require proper handling to remain secure.

Protect Your Private Key: Never share your private key file (`~/.ssh/id_ed25519`). Do not email it, do not commit it to Git, do not upload it to cloud storage. Treat it like a password—keep it secret.

Use Passphrases in Production: For personal projects and learning, a passphrase-free key is convenient. For professional work, use a strong passphrase to protect your private key.

Revoke Compromised Keys: If you suspect your private key has been compromised (e.g., your laptop was stolen), immediately remove the corresponding public key from GitHub. Go to Settings → SSH and GPG keys and delete the key. Then generate a new key pair.

Use Separate Keys for Different Purposes: You can generate multiple key pairs for different uses (e.g., one for work, one for personal projects). This limits the impact if one key is compromised.

Backup Your Keys: If you lose your private key, you cannot recover it. You will need to generate a new key pair and update GitHub. Consider backing up your `~/.ssh` directory to secure storage (encrypted external drive, password manager, etc.).

Part 4: Arduino IoT Sensor Project

Hardware Setup and Wiring

Now we move from software tools to hardware. In this section, you will wire sensors on a breadboard, connect them to an Arduino, and prepare the physical setup for data collection.

Understanding the Components

Before we start wiring, let us understand what each component does.

Arduino Uno is a microcontroller board based on the ATmega328P chip. It has 14 digital input/output pins, 6 analog input pins, a USB connection for programming and power, and a 5V power supply. The Arduino runs your code (called a "sketch") and communicates with sensors and your computer.

Breadboard is a prototyping tool that lets you build circuits without soldering. It has rows of connected holes where you insert component leads and wires. The two long rows on each side are power rails (usually red for +5V and blue/black for ground). The middle rows are for components, with each row of 5 holes connected internally.

DHT11 or DHT22 Temperature and Humidity Sensor measures ambient temperature and humidity. The DHT11 is less accurate (±2°C, ±5% humidity) but cheaper. The DHT22 is more accurate (±0.5°C, ±2% humidity) but slightly more expensive. Both use the same wiring and code. The sensor has three pins: VCC (power), DATA (signal), and GND (ground).

Photoresistor (Light-Dependent Resistor, LDR) is a variable resistor whose resistance decreases as light intensity increases. In darkness, its resistance is high (several

megaohms). In bright light, its resistance is low (a few hundred ohms). We use it in a voltage divider circuit to measure light levels.

10kΩ Resistor is used in the voltage divider circuit with the photoresistor. It provides a reference resistance so that changes in the photoresistor' s resistance produce measurable voltage changes.

Jumper Wires connect components on the breadboard and between the breadboard and Arduino. Male-to-male wires connect breadboard holes. Male-to-female wires connect breadboard holes to Arduino pins.

USB Cable connects the Arduino to your computer for programming and power. It also provides serial communication for data transfer.

Using Circuito.io for Wiring Diagrams

Circuito.io is a free online tool that generates wiring diagrams for Arduino projects. It is incredibly helpful for beginners because it shows exactly where to connect each wire.

Step 1: Go to https://www.circuito.io in your web browser.

Step 2: Click "Start" to begin a new project.

Step 3: Add components to your project:

- Click "Add Component" and search for "Arduino Uno." Add it to your workspace.
- Click "Add Component" and search for "DHT22" (or "DHT11" if you have that sensor). Add it.
- Click "Add Component" and search for "Photoresistor" or "LDR." Add it.

Step 4: Click "Generate Circuit." Circuito.io will automatically generate a wiring diagram showing how to connect everything.

Step 5: Review the diagram. It will show:

- Which Arduino pins to use for each sensor
- How to connect power (5V) and ground (GND)
- Where to place the resistor in the photoresistor circuit

Step 6: (Optional) Download the diagram as a PDF or image for reference while wiring.

Wiring the DHT Sensor

Let us wire the DHT sensor first. The exact pin numbers may vary depending on your Circuito.io diagram, but the general approach is the same.

Step 1: Place the DHT sensor on the breadboard. The sensor usually has three pins (some have four, but one is not used). Orient it so the pins are in separate rows.

Step 2: Connect power (VCC):

- Use a red jumper wire to connect the VCC pin of the DHT sensor to the 5V power rail on the breadboard.
- Use another red jumper wire to connect the 5V power rail to the 5V pin on the Arduino.

Step 3: Connect ground (GND):

- Use a black jumper wire to connect the GND pin of the DHT sensor to the ground rail on the breadboard.
- Use another black jumper wire to connect the ground rail to a GND pin on the Arduino.

Step 4: Connect the data pin:

- Use a colored jumper wire (e.g., yellow or green) to connect the DATA pin of the DHT sensor to a digital pin on the Arduino. Circuito.io will suggest a pin number—commonly pin 2, 3, or 4. Remember this pin number; you will need it in your code.

Wiring the Photoresistor

The photoresistor requires a voltage divider circuit. Here is how to wire it.

Step 1: Place the photoresistor on the breadboard. It has two leads (no polarity, so orientation does not matter).

Step 2: Connect one lead of the photoresistor to the 5V power rail.

Step 3: Connect the other lead of the photoresistor to an analog input pin on the Arduino (e.g., A0). Use a jumper wire for this connection.

Step 4: Place the 10kΩ resistor on the breadboard. Connect one lead to the same row as the photoresistor lead that goes to A0.

Step 5: Connect the other lead of the resistor to the ground rail.

This creates a voltage divider: 5V → Photoresistor → A0 → Resistor → GND. The voltage at A0 varies with the photoresistor's resistance, which varies with light intensity.

Double-Check Your Wiring

Before powering on the Arduino, double-check your wiring:

- **Power Connections:** Verify that 5V goes to 5V and GND goes to GND. Reversing these can damage components.
- **Data Pins:** Verify that sensor data pins are connected to the correct Arduino pins (the ones you will use in your code).
- **No Short Circuits:** Ensure that power and ground are not directly connected (this would create a short circuit).

Once you are confident the wiring is correct, connect the Arduino to your computer via USB. The Arduino should power on (you will see an LED light up on the board).

Writing Arduino Code

Now that the hardware is set up, let us write the Arduino code (called a "sketch") to read sensor data and send it to the computer via serial communication.

Installing the Arduino IDE

If you have not already installed the Arduino IDE (Integrated Development Environment), download it from https://www.arduino.cc/en/software. Install it following the instructions for your operating system.

Launch the Arduino IDE. You will see a blank sketch with two functions:

```
void setup() {
  // put your setup code here, to run once:
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

setup() runs once when the Arduino powers on or resets. Use it for initialization (setting pin modes, starting serial communication, etc.).

loop() runs repeatedly after setup() finishes. Use it for your main program logic (reading sensors, sending data, etc.).

Installing the DHT Library

To read data from the DHT sensor, you need to install the DHT library.

Step 1: In the Arduino IDE, go to **Sketch → Include Library → Manage Libraries**.

Step 2: In the Library Manager, search for "DHT sensor library" by Adafruit.

Step 3: Click **Install**. The IDE will also prompt you to install dependencies (Adafruit Unified Sensor library). Click **Install All**.

Step 4: Close the Library Manager.

Writing the Sensor Reading Code

Now let us write the code to read the DHT sensor and photoresistor and send the data via serial communication.

Create a new sketch (File → New) and replace the default code with the following:

```
#include <DHT.h>

// Define sensor pins
#define DHTPIN 2          // DHT sensor data pin (change if you used a
different pin)
#define DHTTYPE DHT22     // Change to DHT11 if you have that sensor
#define PHOTORESISTOR_PIN A0  // Photoresistor connected to analog pin A0

// Initialize DHT sensor
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  // Start serial communication at 9600 baud
  Serial.begin(9600);

  // Initialize DHT sensor
  dht.begin();

  // Print header for CSV format
  Serial.println("Temperature,Humidity,Light");
}

void loop() {
  // Read temperature and humidity from DHT sensor
  float temperature = dht.readTemperature();  // Celsius
  float humidity = dht.readHumidity();        // Percentage

  // Read light level from photoresistor (0-1023)
  int lightLevel = analogRead(PHOTORESISTOR_PIN);

  // Check if DHT reading failed
  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("ERROR: Failed to read from DHT sensor");
  } else {
    // Print data in CSV format: temperature,humidity,light
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(humidity);
    Serial.print(",");
    Serial.println(lightLevel);
  }

  // Wait 2 seconds before next reading
```

```
  delay(2000);
}
```

Code Explanation:

- **#include :** Includes the DHT library so we can use its functions.
- **#define DHTPIN 2:** Defines the Arduino pin connected to the DHT data pin. Change this if you used a different pin.
- **#define DHTTYPE DHT22:** Specifies the sensor type. Change to DHT11 if you have that sensor.
- **#define PHOTORESISTOR_PIN A0:** Defines the analog pin connected to the photoresistor.
- **DHT dht(DHTPIN, DHTTYPE):** Creates a DHT object for reading the sensor.
- **Serial.begin(9600):** Starts serial communication at 9600 baud (bits per second). This must match the baud rate in your Python code.
- **dht.begin():** Initializes the DHT sensor.
- **Serial.println("Temperature,Humidity,Light"):** Prints a header line in CSV format. This helps identify the data columns.
- **dht.readTemperature():** Reads temperature in Celsius. Use `dht.readTemperature(true)` for Fahrenheit.
- **dht.readHumidity():** Reads relative humidity as a percentage.
- **analogRead(PHOTORESISTOR_PIN):** Reads the analog voltage at A0 and converts it to a value from 0 (0V) to 1023 (5V).
- **isnan(temperature):** Checks if the reading is "Not a Number," which indicates a sensor error.
- **Serial.print() and Serial.println():** Send data over the serial connection. `print()` does not add a newline; `println()` adds a newline at the end.
- **delay(2000):** Waits 2000 milliseconds (2 seconds) before the next reading. The DHT sensor requires at least 2 seconds between readings.

Uploading the Code to Arduino

Step 1: Connect your Arduino to your computer via USB if it is not already connected.

Step 2: In the Arduino IDE, select your board type:

- Go to **Tools → Board** and select **Arduino Uno** (or your specific board model).

Step 3: Select the correct port:

- Go to **Tools → Port** and select the port where your Arduino is connected. On macOS/Linux, it will be something like `/dev/cu.usbmodem14201` or `/dev/ttyUSB0`. On Windows, it will be something like `COM3` or `COM4`.

Step 4: Click the **Upload** button (the right-arrow icon) in the Arduino IDE toolbar.

The IDE will compile your code and upload it to the Arduino. You will see a progress bar and messages like "Compiling sketch…" and "Uploading…". When the upload is complete, you will see "Done uploading."

Step 5: Open the Serial Monitor to verify the code is working:

- Go to **Tools → Serial Monitor** or click the magnifying glass icon in the top-right corner.
- Set the baud rate dropdown (bottom-right of the Serial Monitor) to **9600 baud** (must match the `Serial.begin(9600)` in your code).
- You should see output like:

```
Temperature,Humidity,Light
23.50,45.20,512
23.60,45.10,510
23.50,45.30,515
```

If you see this output, congratulations! Your Arduino is reading sensor data and sending it via serial communication. If you see errors or no output, check the troubleshooting section at the end of this guide.

Part 5: Python Serial Communication

Installing PySerial

To read Arduino data in Python, you need the PySerial library, which provides serial communication functionality.

Step 1: Open a terminal and install PySerial using pip:

```
pip3 install pyserial
```

Step 2: Verify the installation:

```
python3 -c "import serial; print(serial.VERSION)"
```

You should see a version number like `3.5` or similar.

Finding the Arduino Serial Port

Before you can read data from the Arduino, you need to know which serial port it is connected to.

On macOS/Linux:

List all serial ports:

```
ls /dev/tty.*
```

Look for a port like `/dev/tty.usbmodem14201` or `/dev/ttyUSB0`. This is your Arduino's port.

On Windows:

Open Device Manager (Win+X → Device Manager) and expand "Ports (COM & LPT)". Look for "Arduino Uno (COM3)" or similar. Note the COM port number (e.g., `COM3`).

Alternative Method:

Open the Arduino IDE, go to Tools → Port, and note the port that is selected. This is the port your Arduino is using.

Writing Python Code to Read Serial Data

Now let us write Python code to read data from the Arduino and print it to the console.

Create a new Python file in your project directory:

```
cd ~/arduino-iot-workshop
nano read_arduino.py
```

Add the following code (replace `/dev/tty.usbmodem14201` with your actual port):

```
import serial
import time

# Configure serial connection
# Replace '/dev/tty.usbmodem14201' with your Arduino's port
# On Windows, use 'COM3' or similar
SERIAL_PORT = '/dev/tty.usbmodem14201'
BAUD_RATE = 9600

# Open serial connection
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)

# Wait for Arduino to reset (it resets when serial connection opens)
time.sleep(2)

print("Reading data from Arduino...")
print("Press Ctrl+C to stop")

try:
    while True:
        # Read one line from serial port
        line = ser.readline().decode('utf-8').strip()

        # Skip empty lines
        if line:
            print(line)

except KeyboardInterrupt:
    print("\nStopping...")

finally:
    # Close serial connection
    ser.close()
    print("Serial connection closed.")
```

Save the file and run it:

```
python3 read_arduino.py
```

You should see output like:

```
Reading data from Arduino...
Press Ctrl+C to stop
Temperature,Humidity,Light
23.50,45.20,512
23.60,45.10,510
23.50,45.30,515
```

Press Ctrl+C to stop the script.

Code Explanation:

- **import serial:** Imports the PySerial library.
- **SERIAL_PORT:** The port where your Arduino is connected. Change this to match your system.
- **BAUD_RATE:** Must match the baud rate in your Arduino code (9600).
- **serial.Serial():** Opens a serial connection with a 1-second timeout.
- **time.sleep(2):** Waits 2 seconds for the Arduino to reset. When you open a serial connection, the Arduino resets automatically.
- **ser.readline():** Reads one line from the serial port (up to a newline character).
- **decode('utf-8'):** Converts bytes to a string.
- **strip():** Removes leading/trailing whitespace and newline characters.
- **KeyboardInterrupt:** Catches Ctrl+C so we can close the serial connection gracefully.
- **ser.close():** Closes the serial connection.

Parsing and Storing Sensor Data

Now let us enhance the script to parse the CSV data and store it in a list for later visualization.

Modify `read_arduino.py`:

```python
import serial
import time

# Configure serial connection
SERIAL_PORT = '/dev/tty.usbmodem14201'  # Change to your port
BAUD_RATE = 9600

# Open serial connection
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)

# Data storage
data = {
    'temperature': [],
    'humidity': [],
    'light': []
}

print("Reading data from Arduino...")
print("Press Ctrl+C to stop")

try:
    # Skip the header line
    header = ser.readline().decode('utf-8').strip()
    print(f"Header: {header}")

    while True:
        # Read one line
        line = ser.readline().decode('utf-8').strip()

        # Skip empty lines
        if not line:
            continue

        # Parse CSV data
        try:
            temp, humidity, light = line.split(',')
            temp = float(temp)
            humidity = float(humidity)
            light = int(light)

            # Store data
            data['temperature'].append(temp)
            data['humidity'].append(humidity)
            data['light'].append(light)
```

```
            # Print current reading
            print(f"Temp: {temp}°C, Humidity: {humidity}%, Light: {light}")

        except ValueError:
            print(f"Error parsing line: {line}")

except KeyboardInterrupt:
    print("\nStopping...")

finally:
    ser.close()
    print(f"Collected {len(data['temperature'])} readings.")
    print("Serial connection closed.")
```

Run the script again:

```
python3 read_arduino.py
```

Let it run for 30-60 seconds to collect data, then press Ctrl+C. You will see a summary of how many readings were collected.

Creating Real-Time Visualizations with Matplotlib

Now let us create a real-time visualization that updates as new data arrives. This is where the "wow factor" comes in—you will see live plots updating as the Arduino sends data.

Create a new file:

```
nano visualize_realtime.py
```

Add this code:

```python
import serial
import time
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import deque

# Configure serial connection
SERIAL_PORT = '/dev/tty.usbmodem14201'  # Change to your port
BAUD_RATE = 9600

# Open serial connection
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)

# Data storage (using deque for efficient appending and limiting size)
MAX_DATA_POINTS = 50
data = {
    'temperature': deque(maxlen=MAX_DATA_POINTS),
    'humidity': deque(maxlen=MAX_DATA_POINTS),
    'light': deque(maxlen=MAX_DATA_POINTS),
    'time': deque(maxlen=MAX_DATA_POINTS)
}

# Skip header line
ser.readline()

# Create figure with three subplots
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 8))
fig.suptitle('Real-Time Arduino Sensor Data', fontsize=16)

# Initialize empty lines
line1, = ax1.plot([], [], 'r-', label='Temperature')
line2, = ax2.plot([], [], 'b-', label='Humidity')
line3, = ax3.plot([], [], 'g-', label='Light Level')

# Configure axes
ax1.set_ylabel('Temperature (°C)')
ax1.legend(loc='upper right')
ax1.grid(True)

ax2.set_ylabel('Humidity (%)')
ax2.legend(loc='upper right')
ax2.grid(True)

ax3.set_xlabel('Time (seconds)')
ax3.set_ylabel('Light Level')
```

```
ax3.legend(loc='upper right')
ax3.grid(True)

# Start time
start_time = time.time()

def update_plot(frame):
    """Read new data and update plots."""
    try:
        # Read one line
        line = ser.readline().decode('utf-8').strip()

        if line:
            # Parse CSV data
            temp, humidity, light = line.split(',')
            temp = float(temp)
            humidity = float(humidity)
            light = int(light)

            # Calculate elapsed time
            elapsed = time.time() - start_time

            # Append data
            data['temperature'].append(temp)
            data['humidity'].append(humidity)
            data['light'].append(light)
            data['time'].append(elapsed)

            # Update line data
            line1.set_data(data['time'], data['temperature'])
            line2.set_data(data['time'], data['humidity'])
            line3.set_data(data['time'], data['light'])

            # Adjust axis limits
            if len(data['time']) > 0:
                ax1.set_xlim(min(data['time']), max(data['time']))
                ax2.set_xlim(min(data['time']), max(data['time']))
                ax3.set_xlim(min(data['time']), max(data['time']))

                ax1.set_ylim(min(data['temperature']) - 1,
max(data['temperature']) + 1)
                ax2.set_ylim(min(data['humidity']) - 5,
max(data['humidity']) + 5)
                ax3.set_ylim(min(data['light']) - 50, max(data['light']) +
50)

    except Exception as e:
```

```
        print(f"Error: {e}")

    return line1, line2, line3

# Create animation
ani = animation.FuncAnimation(fig, update_plot, interval=100, blit=True)

# Show plot
plt.tight_layout()
plt.show()

# Close serial connection when plot is closed
ser.close()
print("Serial connection closed.")
```

Run the script:

```
python3 visualize_realtime.py
```

You will see a window with three plots that update in real-time as the Arduino sends data. Try covering the photoresistor with your hand to see the light level drop. Breathe on the DHT sensor to see temperature and humidity change.

This is the "wow factor" moment—you have built a complete IoT data pipeline from hardware to real-time visualization!

Code Explanation:

- **deque(maxlen=MAX_DATA_POINTS):** A double-ended queue that automatically removes old data when it reaches the maximum size. This prevents memory issues during long-running visualizations.
- **plt.subplots(3, 1):** Creates a figure with three subplots arranged vertically.
- **animation.FuncAnimation():** Calls the `update_plot()` function repeatedly to update the plots.
- **interval=100:** Updates every 100 milliseconds (10 times per second).
- **blit=True:** Optimizes rendering by only redrawing changed elements.
- **set_xlim() and set_ylim():** Dynamically adjust axis limits as new data arrives.

Part 6: Weather API Integration

Obtaining an OpenWeatherMap API Key

To enrich your sensor data with external weather information, you will use the OpenWeatherMap API. This free API provides current weather data for any location.

Step 1: Go to https://openweathermap.org/api and click "Sign Up" to create a free account.

Step 2: After signing up and verifying your email, log in and go to your account dashboard.

Step 3: Click on "API keys" in the menu. You will see a default API key already generated.

Step 4: Copy your API key (a long string of characters). Store it securely—you will use it in your Python code.

Note: The free tier allows 1,000 API calls per day, which is more than sufficient for this workshop.

Fetching Weather Data with Python

Create a new Python file to test the API:

```
nano fetch_weather.py
```

Add this code (replace `YOUR_API_KEY` with your actual API key):

```python
import requests

# Configuration
API_KEY = 'YOUR_API_KEY'
CITY = 'London'  # Change to your city
API_URL = f'https://api.openweathermap.org/data/2.5/weather?q={CITY}&appid=
{API_KEY}&units=metric'

# Fetch weather data
response = requests.get(API_URL)

if response.status_code == 200:
    data = response.json()

    # Extract relevant information
    temp = data['main']['temp']
    humidity = data['main']['humidity']
    description = data['weather'][0]['description']

    print(f"Weather in {CITY}:")
    print(f"Temperature: {temp}°C")
    print(f"Humidity: {humidity}%")
    print(f"Description: {description}")
else:
    print(f"Error: {response.status_code}")
    print(response.text)
```

Run the script:

```
python3 fetch_weather.py
```

You should see output like:

```
Weather in London:
Temperature: 15.3°C
Humidity: 72%
Description: light rain
```

Combining Arduino and API Data

Now let us create a script that combines Arduino sensor data (indoor conditions) with OpenWeatherMap data (outdoor conditions) and visualizes both.

Create a new file:

```
nano combined_visualization.py
```

Add this code:

```python
import serial
import time
import requests
import matplotlib.pyplot as plt
from collections import deque

# Configuration
SERIAL_PORT = '/dev/tty.usbmodem14201'  # Change to your port
BAUD_RATE = 9600
API_KEY = 'YOUR_API_KEY'  # Change to your API key
CITY = 'London'  # Change to your city
API_URL = f'https://api.openweathermap.org/data/2.5/weather?q={CITY}&appid=
{API_KEY}&units=metric'

# Open serial connection
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)
ser.readline()  # Skip header

# Data storage
MAX_POINTS = 30
indoor_temp = deque(maxlen=MAX_POINTS)
indoor_humidity = deque(maxlen=MAX_POINTS)
timestamps = deque(maxlen=MAX_POINTS)

# Fetch outdoor weather data
def get_outdoor_weather():
    try:
        response = requests.get(API_URL)
        if response.status_code == 200:
            data = response.json()
            return data['main']['temp'], data['main']['humidity']
    except Exception as e:
        print(f"Error fetching weather: {e}")
    return None, None

# Get initial outdoor data
outdoor_temp, outdoor_humidity = get_outdoor_weather()

# Create figure
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle('Indoor (Arduino) vs Outdoor (API) Conditions', fontsize=16)

start_time = time.time()

print("Collecting data... Press Ctrl+C to stop and show final comparison.")
```

```
try:
    while True:
        # Read Arduino data
        line = ser.readline().decode('utf-8').strip()

        if line:
            temp, humidity, light = line.split(',')
            temp = float(temp)
            humidity = float(humidity)

            indoor_temp.append(temp)
            indoor_humidity.append(humidity)
            timestamps.append(time.time() - start_time)

            print(f"Indoor: {temp}°C, {humidity}% | Outdoor: {outdoor_temp}
°C, {outdoor_humidity}%")

except KeyboardInterrupt:
    print("\nGenerating final comparison plot...")

finally:
    ser.close()

    # Calculate averages
    avg_indoor_temp = sum(indoor_temp) / len(indoor_temp) if indoor_temp
else 0
    avg_indoor_humidity = sum(indoor_humidity) / len(indoor_humidity) if
indoor_humidity else 0

    # Temperature comparison
    ax1.bar(['Indoor\n(Arduino)', 'Outdoor\n(API)'],
            [avg_indoor_temp, outdoor_temp],
            color=['#FF6B35', '#004E89'])
    ax1.set_ylabel('Temperature (°C)')
    ax1.set_title('Temperature Comparison')
    ax1.grid(axis='y', alpha=0.3)

    # Humidity comparison
    ax2.bar(['Indoor\n(Arduino)', 'Outdoor\n(API)'],
            [avg_indoor_humidity, outdoor_humidity],
            color=['#00D4FF', '#00A896'])
    ax2.set_ylabel('Humidity (%)')
    ax2.set_title('Humidity Comparison')
    ax2.grid(axis='y', alpha=0.3)

    plt.tight_layout()
```

```
    plt.savefig('indoor_outdoor_comparison.png', dpi=150)
    print("Comparison saved as 'indoor_outdoor_comparison.png'")
    plt.show()
```

Run the script:

```
python3 combined_visualization.py
```

Let it collect data for 1-2 minutes, then press Ctrl+C. You will see a bar chart comparing indoor (Arduino) and outdoor (API) temperature and humidity. The chart is also saved as an image file.

This demonstrates how you can combine real-time sensor data with external API data to create richer insights.

Part 7: Containerizing Your Application

Writing a Dockerfile for the Arduino Project

Now that you have a working Python application that reads Arduino data and creates visualizations, let us containerize it with Docker. This ensures that anyone can run your project with identical results, regardless of their system.

Create a Dockerfile in your project directory:

```
cd ~/arduino-iot-workshop
nano Dockerfile
```

Add this content:

```
# Use Python 3.9 slim image as base
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements file
COPY requirements.txt /app/

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY *.py /app/

# Set environment variable to prevent Python buffering
ENV PYTHONUNBUFFERED=1

# Default command (can be overridden)
CMD ["python", "read_arduino.py"]
```

Now create a `requirements.txt` file listing all Python dependencies:

```
nano requirements.txt
```

Add:

```
pyserial==3.5
matplotlib==3.5.3
requests==2.28.1
```

Save the file.

Building the Docker Image

Build the Docker image:

```
docker build -t arduino-iot-app .
```

This will take a few minutes the first time as Docker downloads the base image and installs dependencies. Subsequent builds will be much faster due to layer caching.

Running the Container with Serial Port Access

Running a Docker container that needs access to a serial port (the Arduino) requires special permissions.

On macOS/Linux:

```
docker run --rm -it --device=/dev/tty.usbmodem14201:/dev/ttyUSB0 arduino-iot-app
```

Replace `/dev/tty.usbmodem14201` with your Arduino's port. The `--device` flag maps the host's serial port to `/dev/ttyUSB0` inside the container.

On Windows:

Serial port access from Docker on Windows is more complex and may require WSL 2 configuration. For simplicity during the workshop, you can run the Python scripts directly on Windows and use Docker for deployment on Linux servers.

Explanation of Docker Run Flags:

- `--rm`: Automatically remove the container when it stops
- `-it`: Interactive terminal (allows you to see output and press Ctrl+C to stop)
- `--device`: Maps a host device to the container

Verifying Reproducibility

The beauty of Docker is that your containerized application will run identically on any system. To verify this, you can:

1. **Share your Docker image** with a colleague by pushing it to Docker Hub
2. **Run the container on a different computer** and verify it produces the same results
3. **Run the container six months from now** and it will still work, even if you have upgraded Python or libraries on your host system

This is 100% reproducibility—the core benefit of Docker.

Part 8: Git Workflow and Collaboration

Initializing a Git Repository

Now that you have a complete project, let us put it under version control with Git.

Step 1: Initialize a Git repository in your project directory:

```
cd ~/arduino-iot-workshop
git init
```

You will see:

```
Initialized empty Git repository in /Users/yourname/arduino-iot-
workshop/.git/
```

Step 2: Create a `.gitignore` file to exclude files that should not be committed:

```
nano .gitignore
```

Add:

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
env/
venv/

# Data files
*.csv
*.png
*.jpg

# API keys (never commit these!)
config.py
.env

# OS files
.DS_Store
Thumbs.db
```

Save the file.

Step 3: Add all files to the staging area:

```
git add .
```

Step 4: Create your first commit:

```
git commit -m "Initial commit: Arduino IoT sensor project with Python visualization"
```

Creating a GitHub Repository

Step 1: Go to https://github.com and sign in.

Step 2: Click the "+" icon in the top-right corner and select "New repository."

Step 3: Enter a repository name (e.g., `arduino-iot-workshop`), add a description, and choose "Public" or "Private."

Step 4: Do NOT initialize with a README, .gitignore, or license (you already have these locally).

Step 5: Click "Create repository."

Step 6: GitHub will show you commands to push an existing repository. Copy the SSH URL (it looks like `git@github.com:yourusername/arduino-iot-workshop.git`).

Step 7: Add the remote repository and push your code:

```
git remote add origin git@github.com:yourusername/arduino-iot-workshop.git
git branch -M main
git push -u origin main
```

Your code is now on GitHub!

Creating Meaningful Commits

Good commit messages are essential for collaboration and understanding project history. Follow these best practices:

Use the Imperative Mood: "Add feature" not "Added feature" or "Adds feature."

Be Specific: "Fix serial port timeout issue" not "Fix bug."

Explain Why, Not Just What: If the change is not obvious, add a longer description:

```
git commit -m "Add error handling for DHT sensor failures" -m "The DHT
sensor occasionally returns NaN values. This commit adds checks to skip
invalid readings and log errors instead of crashing."
```

Commit Often: Small, focused commits are better than large, monolithic commits.

Branching and Pull Requests

For collaborative work, use feature branches and pull requests.

Step 1: Create a new branch for a feature:

```
git checkout -b add-ml-predictions
```

Step 2: Make changes, commit them:

```
# ... edit files ...
git add .
git commit -m "Add machine learning temperature prediction"
```

Step 3: Push the branch to GitHub:

```
git push -u origin add-ml-predictions
```

Step 4: On GitHub, you will see a prompt to "Compare & pull request." Click it.

Step 5: Write a description of your changes and click "Create pull request."

Step 6: Team members can review your code, leave comments, and approve the pull request.

Step 7: Once approved, merge the pull request into the main branch.

This workflow ensures that all code is reviewed before being merged, improving code quality and catching bugs early.

Handling Merge Conflicts

Merge conflicts occur when two people edit the same lines of code. Here is how to resolve them:

Step 1: Pull the latest changes:

```
git pull origin main
```

If there is a conflict, Git will notify you:

```
CONFLICT (content): Merge conflict in read_arduino.py
Automatic merge failed; fix conflicts and then commit the result.
```

Step 2: Open the conflicted file. You will see conflict markers:

```
<<<<<<< HEAD
SERIAL_PORT = '/dev/tty.usbmodem14201'
=======
SERIAL_PORT = '/dev/ttyUSB0'
>>>>>>> add-ml-predictions
```

Step 3: Edit the file to resolve the conflict. Decide which version to keep (or combine both). Remove the conflict markers.

Step 4: Stage the resolved file:

```
git add read_arduino.py
```

Step 5: Complete the merge:

```
git commit
```

Git will create a merge commit automatically.

Optional Extensions

Machine Learning Predictions

Extend your project by training a machine learning model to predict future sensor readings.

Approach: Collect sensor data over several hours or days. Use scikit-learn or TensorFlow to train a time series prediction model (e.g., LSTM or ARIMA). Predict

temperature and humidity for the next hour based on historical data.

Resources:

- scikit-learn time series tutorial: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- TensorFlow time series guide: https://www.tensorflow.org/tutorials/structured_data/time_series

Real-Time Dashboard

Build an interactive web dashboard using Flask or Dash.

Approach: Create a Flask web server that reads Arduino data and serves it via a REST API. Build a frontend with HTML/JavaScript that fetches data from the API and displays live plots using Chart.js or Plotly.

Resources:

- Flask tutorial: https://flask.palletsprojects.com/en/2.3.x/quickstart/
- Dash tutorial: https://dash.plotly.com/tutorial

Cloud Deployment

Deploy your containerized application to a cloud platform.

Approach: Push your Docker image to Docker Hub. Deploy it to AWS (using ECS or EC2), Azure (using Container Instances), or Google Cloud (using Cloud Run). Configure the cloud instance to access a remote Arduino via network or use cloud-based IoT services.

Resources:

- AWS ECS tutorial: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/getting-started.html
- Azure Container Instances: https://learn.microsoft.com/en-us/azure/container-instances/
- Google Cloud Run: https://cloud.google.com/run/docs/quickstarts

Multi-Sensor Network

Connect multiple Arduino boards with different sensors and aggregate their data.

Approach: Set up multiple Arduinos, each with a unique serial port. Modify your Python script to read from multiple ports simultaneously (using threading or multiprocessing). Aggregate data in a central database (SQLite or PostgreSQL) and visualize it.

Resources:

- Python threading: https://docs.python.org/3/library/threading.html
- SQLite tutorial: https://docs.python.org/3/library/sqlite3.html

Generative Art

Create data-driven visualizations that respond to sensor data.

Approach: Use p5.js or Processing to create generative art where colors, shapes, and animations change based on temperature, humidity, and light levels. For example, warmer temperatures could shift colors toward red, higher humidity could increase particle density, and brighter light could make shapes larger.

Resources:

- p5.js tutorial: https://p5js.org/learn/
- Processing tutorial: https://processing.org/tutorials

Troubleshooting Guide

AI Terminal Issues

Problem: `gemini: command not found`

Solution: The npm global bin directory is not in your PATH. Find the directory with `npm config get prefix` and add `<prefix>/bin` to your PATH in your shell configuration file.

Problem: API key not working

Solution: Verify the key is correct by checking `echo $GEMINI_API_KEY`. Ensure you reloaded your shell configuration (`source ~/.bashrc` or `source ~/.zshrc`). Regenerate the API key if necessary.

Problem: Context file not created

Solution: Ensure you are running `gemini` from a directory where you have write permissions. Check that the directory is not read-only.

Docker Issues

Problem: Permission denied when running Docker commands

Solution: On Linux, add your user to the docker group: `sudo usermod -aG docker $USER`, then log out and log back in.

Problem: Port already in use

Solution: Find the process using the port with `lsof -i :PORT` (macOS/Linux) or `netstat -ano | findstr :PORT` (Windows). Kill the process or use a different port.

Problem: Image build fails

Solution: Check Dockerfile syntax. Ensure the base image exists (try `docker pull python:3.9-slim` manually). Clear Docker cache with `docker build --no-cache`.

Problem: Container exits immediately

Solution: Check logs with `docker logs CONTAINER_ID`. Verify the CMD instruction in your Dockerfile is correct. Ensure the script does not exit immediately (e.g., add a loop or wait).

GitHub SSH Issues

Problem: Permission denied (publickey)

Solution: Verify your public key is added to GitHub (Settings → SSH and GPG keys). Test the connection with `ssh -T git@github.com`. Ensure the SSH agent is running and your key is added (`ssh-add -l`).

Problem: Key not added to agent

Solution: Start the agent with `eval "$(ssh-agent -s)"` and add the key with `ssh-add ~/.ssh/id_ed25519`.

Problem: Wrong repository URL

Solution: Verify you are using the SSH URL format (`git@github.com:user/repo.git`), not HTTPS. Change the remote URL with `git remote set-url origin git@github.com:user/repo.git`.

Arduino Issues

Problem: Port not found

Solution: Check the USB cable connection. Install Arduino drivers if necessary (usually automatic on modern systems). Verify the port with `ls /dev/tty.*` (macOS/Linux) or Device Manager (Windows).

Problem: Upload failed

Solution: Ensure the correct board and port are selected in the Arduino IDE (Tools → Board and Tools → Port). Close the Serial Monitor before uploading (it locks the port). Try a different USB cable or port.

Problem: Sensor not reading

Solution: Double-check wiring against the Circuito.io diagram. Verify power (5V) and ground (GND) connections. Test the sensor with example code from the library (File → Examples → DHT sensor library → DHTtester).

Problem: Serial monitor blank

Solution: Ensure the baud rate in the Serial Monitor matches the `Serial.begin()` value in your code (9600). Reset the Arduino by pressing the reset button. Check that `Serial.println()` statements are in your code.

Resources for Continued Learning

AI in Terminal

- **NetworkChuck Video Tutorial:** https://www.youtube.com/watch?v=MsQACpcuTkU
- **GitHub Repository:** https://github.com/theNetworkChuck/ai-in-the-terminal
- **Google Gemini AI Documentation:** https://ai.google.dev/gemini-api/docs

Docker

- **Official Docker Documentation:** https://docs.docker.com
- **Docker Hub (Find Images):** https://hub.docker.com
- **Docker Tutorial for Beginners:** https://docker-curriculum.com
- **Best Practices Guide:** https://docs.docker.com/develop/dev-best-practices/

GitHub & Git

- **SSH Key Setup Guide:** https://docs.github.com/en/authentication/connecting-to-github-with-ssh
- **GitHub Skills (Interactive Tutorials):** https://skills.github.com
- **Pro Git Book (Free):** https://git-scm.com/book/en/v2

Arduino

- **Arduino Language Reference:** https://www.arduino.cc/reference/en/
- **DHT Sensor Library Documentation:** https://github.com/adafruit/DHT-sensor-library
- **Circuito.io (Wiring Diagrams):** https://www.circuito.io
- **Arduino Project Hub:** https://create.arduino.cc/projecthub

Data Science

- **Python Data Science Handbook:** https://jakevdp.github.io/PythonDataScienceHandbook/
- **Matplotlib Documentation:** https://matplotlib.org/stable/tutorials/
- **Pandas User Guide:** https://pandas.pydata.org/docs/user_guide/
- **Kaggle Learn (Free Courses):** https://www.kaggle.com/learn
- **Real-Time Data Visualization:** https://matplotlib.org/stable/users/explain/animations/

Conclusion

Congratulations on completing this comprehensive workshop! You have built a complete IoT data pipeline that integrates AI-assisted coding, Docker containerization, GitHub collaboration, and Arduino hardware. You now have the professional skills that appear in 87% of data science job postings and are essential for modern research and industry work.

What You Accomplished:

- **AI Terminal Integration:** You installed and used Gemini CLI for terminal-based AI assistance with persistent context and direct file access, achieving a 10x faster workflow compared to browser-based AI tools.
- **Docker Mastery:** You learned Docker concepts, created Dockerfiles, built images, and ran containers, achieving 100% reproducibility for your data science projects.
- **GitHub SSH Authentication:** You generated SSH key pairs, configured public-key authentication, and eliminated password prompts for Git operations.
- **Arduino IoT Sensors:** You wired sensors on a breadboard, wrote Arduino code, and established serial communication between hardware and Python.
- **Python Data Pipelines:** You read serial data, parsed CSV output, and created real-time matplotlib visualizations of sensor data.
- **API Integration:** You fetched weather data from OpenWeatherMap and combined it with Arduino sensor data for comparative analysis.

- **Professional Workflow:** You initialized Git repositories, created meaningful commits, pushed to GitHub, and learned branching and pull request workflows.

Next Steps:

- **Extend Your Project:** Implement one or more of the optional extensions (machine learning, dashboards, cloud deployment, multi-sensor networks, generative art).
- **Build Your Portfolio:** Clean up your GitHub repository with a comprehensive README, documentation, and example outputs. This is a portfolio piece that demonstrates your skills to employers and collaborators.
- **Share Your Work:** Create a social media post (LinkedIn, Twitter) with a video or images of your project. Use hashtags like #DataScience, #IoT, #Arduino, #Docker, #MachineLearning to reach a wider audience.
- **Contribute to Open Source:** Find Arduino or data science projects on GitHub and contribute improvements, documentation, or bug fixes. This builds your reputation and network in the community.
- **Keep Learning:** Technology evolves rapidly. Stay curious, experiment with new tools, and keep building projects. The skills you learned today are foundational, and you can build on them throughout your career.

Thank you for participating in this workshop. You now have the tools to build amazing things. Go create something incredible!

Author: Manus AI
Date: November 2025
License: MIT License - Free to use, modify, and distribute with attribution